

# ADEPT: A Unified Framework for Deep Learning Test Adequacy

Yidi Kao<sup>✉</sup>

Auburn University  
Department of Computer Science and Software  
Engineering  
Auburn, USA  
[yidik@auburn.edu](mailto:yidik@auburn.edu)

Tommi Rose Fahy

Auburn University  
Department of Computer Science and Software  
Engineering  
Auburn, USA  
[tommirose.fahy@auburn.edu](mailto:tommirose.fahy@auburn.edu)

Shawn Burnham

Auburn University  
Department of Computer Science and Software  
Engineering  
Auburn, USA  
[shawnburnham@auburn.edu](mailto:shawnburnham@auburn.edu)

Ali Ghanbari

Auburn University  
Department of Computer Science and Software  
Engineering  
Auburn, USA  
[ghanbari@auburn.edu](mailto:ghanbari@auburn.edu)

## Abstract

Over the past decade, many test adequacy metrics have been proposed for deep learning that characterize test dataset adequacy from different perspectives, e.g., neuron activation behavior, latent feature coverage, decision-boundary exploration, etc. However, these metrics are typically released as independent research prototypes with substantially different installation and preprocessing requirements, execution workflows, and configuration mechanisms. These complications make them quite difficult to reproduce, compare, and adopt in research work and practical deployment alike.

In this paper, we present the engineering details of ADEPT, a framework that integrates representative adequacy techniques, including neuron-coverage-based metrics, surprise adequacy, input distribution coverage, boundary coverage, and source- and model-level mutation score, under a consistent execution workflow. ADEPT provides a template-based metric interface with well-defined extension points for integrating new adequacy metrics. Furthermore, it provides YAML-based configuration management, preprocessing-cache reuse, and structured result reporting, making it easy to use in any research and development workflows. ADEPT is designed for researchers and practitioners who wish to reproduce and apply adequacy metrics without spending days or weeks implementing missing tooling or configuring disparate research prototypes. A demo video is available at [https://aub.ie/ADEPT\\_video](https://aub.ie/ADEPT_video).

**CCS Concepts:** • Software and its engineering → Software testing and debugging; • Computing methodologies → Machine learning.

**Keywords:** Deep Learning, Deep Neural Network, Testing, Adequacy, Metric

## ACM Reference Format:

Yidi Kao, Shawn Burnham, Tommi Rose Fahy, and Ali Ghanbari. 2026. ADEPT: A Unified Framework for Deep Learning Test Adequacy. In *Companion Proceedings of the 2026 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity (SPLASH Companion '26)*, October 3–9, 2026, Oakland, CA, USA. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3837729.3840488>

## 1 Introduction

Deep learning [10] (DL) models are increasingly deployed in safety- and mission-critical domains such as autonomous driving, health care, and energy, warranting research to ensure their quality in terms of accuracy, robustness, or efficiency [16]. Among existing quality assurance techniques for DL models, testing remains one of the most widely used approaches in practice [4, 19].

However, good performance on a test dataset does not guarantee model accuracy, robustness, or efficiency on novel inputs. Therefore, there is a need to systematically assess the quality of test datasets themselves to understand how thoroughly they evaluate the model. Additionally, by quantifying test dataset quality, one can compare datasets to each other and pick one dataset over another in a given scenario. For example, practitioners often face large pools of candidate test inputs but limited labeling budgets and computational resources. As a result, they need methods to identify a small subset of inputs that can adequately evaluate the model, allowing them to focus their effort on that subset.



This work is licensed under a Creative Commons Attribution 4.0 International License.

SPLASH Companion '26, Oakland, CA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2910-2/2026/10

<https://doi.org/10.1145/3837729.3840488>

Consequently, over the past decade, researchers have proposed numerous DNN test adequacy metrics to assess test-set quality and support applications such as test dataset prioritization and evaluation. These metrics include, but are not limited to, neuron coverage [15] and its variations [12], surprise adequacy [8], input distribution coverage [2], decision-boundary coverage [11], and source- and model-based mutation score [5, 6].

Despite the large number of adequacy metrics, existing implementations remain highly fragmented. When the implementation is available, different approaches rely on substantially different preprocessing pipelines, execution workflows, and setup requirements. For example, some metrics based on neuron coverage, like KMNC [12], require activation profiling over training data, surprise adequacy relies on activation trace extraction and density estimation, input distribution coverage depends on separately trained encoder models, and mutation testing typically requires generating and executing a large collection of mutants. As a result, reproducing, comparing, and deploying these adequacy metrics in practice often requires substantial effort, either to re-implement unavailable metrics or to configure and run available ones. Case in point, recent interviews with industrial DNN practitioners reveal that existing testing metrics and tools are difficult to use in practice due to heterogeneous workflows, high adoption barriers, limited tool integration, and the absence of publicly available implementations for some techniques [18].

To address this problem, we present ADEPT, an extensible framework for running DL test adequacy metrics. ADEPT provides a unified command-line interface for executing representative adequacy metrics, including neuron coverage and its variants, surprise adequacy, input distribution coverage, decision boundary coverage, and source-/model-level mutation score. The framework is implemented in Python and supports Keras-based DL models. Its unified architecture implements each adequacy metric as an independent module with consistent interfaces for preprocessing and scoring while preserving metric-specific execution logic, allowing users to easily integrate new metrics into the framework. To improve efficiency under repeated evaluations, the framework also supports reusable caching of intermediate artifacts generated during preprocessing, which reduces redundant computation when metrics are repeatedly executed. In this paper, we present the engineering details of ADEPT and show how it can be used.

ADEPT is publicly available at <https://zenodo.org/records/21682100> and a demo video can be found at [https://aub.ie/ADEPT\\_video](https://aub.ie/ADEPT_video).

## 2 ADEPT Overview

To address implementation fragmentation, metric-specific execution workflows, and the lack of public code for some

representative metrics, we design ADEPT, a unified framework for assessing the adequacy of DL test datasets.

Figure 1 presents the overall architecture of ADEPT, which integrates representative adequacy metrics under a common workflow while preserving their metric-specific processing requirements.

The framework consists of four main components: (1) a template-based metric interface that provides access to representative adequacy metrics and enables new metrics to be integrated through well-defined extension points, (2) metric-specific processing modules that perform the preprocessing required by each adequacy technique and generate the corresponding intermediate artifacts, (3) a cache management component that stores and retrieves reusable intermediate artifacts across repeated evaluations, and (4) an adequacy scoring component that computes the final adequacy score for a given test dataset and generates a final report containing the score and execution metadata.

### 2.1 Unified Metric Interface

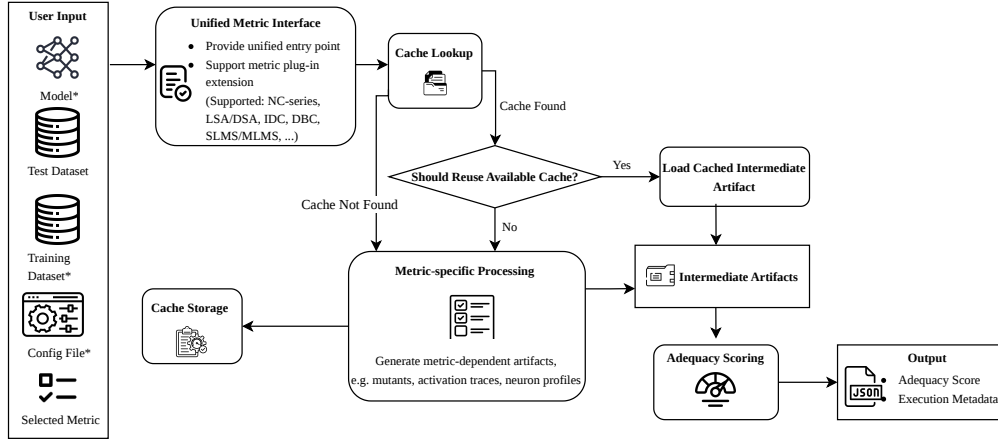
The metric interface provides a unified entry point for executing different DL test dataset adequacy metrics. Each metric is implemented as a plug-in module that follows the same framework-level abstraction while preserving its own processing and scoring logic. This design allows ADEPT to support different adequacy techniques without requiring users to coordinate their different execution requirements manually. New metrics can also be incorporated by implementing a metric module that follows the same template-based interface.

ADEPT integrates several representative DL test adequacy techniques that have been proposed in prior works. Currently, the framework supports neuron coverage and its variations (NC-series), two flavors of surprise adequacy (LSA and DSA), input distribution coverage (IDC), deep boundary coverage (DBC), and source- and model-level mutation score (SLMS and MLMS).

NC-series metrics measure adequacy based on neuron activation behavior observed during model execution. NC and Top-k Neuron Coverage (TKNC) assess whether individual neurons are activated above a threshold or ranked among the top-k most active neurons for a given input.  $k$ -multisection Neuron Coverage (KMNC), Neuron Boundary Coverage (NBC), and Strong Neuron Activation Coverage (SNAC) characterize coverage relative to the activation ranges observed on the training set.

LSA and DSA assess test adequacy by measuring how surprising test inputs are relative to the training data based on their activation traces.

IDC measures adequacy in a learned latent feature space. It employs a variational autoencoder [3, 9] (VAE) to construct a compact representation of the input distribution and applies combinatorial interaction testing to assess feature diversity within a test dataset.



**Figure 1.** Overview of ADEPT. Inputs marked with an asterisk (\*) are optional and are required only by specific metrics.

DBC measures test adequacy from a decision-space perspective, focusing on how test inputs cover the model’s decision boundaries.

SLMS and MLMS evaluate test datasets based on their ability to distinguish mutant models from the original model. The underlying intuition is that more effective test datasets should be capable of exposing behavioral differences introduced by injected faults. ADEPT supports both MLMS, which applies mutation operators directly to trained DL models, and SLMS, which introduces faults before the training process to generate mutants.

Each adequacy metric is accessible through a consistent execution interface. Users specify the test dataset, target model, training dataset if needed, and adequacy metric, while ADEPT automatically invokes the corresponding processing pipeline. This reduces the effort required to use different adequacy techniques.

## 2.2 Metric-Specific Processing

Different adequacy metrics require different forms of pre-processing before adequacy scores can be computed. To support metric-specific requirements and improve extensibility, ADEPT implements each metric’s preprocessing workflow as a dedicated processing module.

For NC-based metrics that rely on training-set activation statistics, i.e., KMNC, NBC, and SNAC, ADEPT performs neuron profiling on the training dataset. For LSA and DSA, activation traces are extracted and used for surprise estimation. IDC relies on pre-trained VAE models to construct latent feature representations, while DBC performs decision-boundary extraction to characterize model behavior near classification boundaries. ADEPT carries out MLMS à la DeepMutation++ [5] and SLMS à la DeepCrime [6].

Although these preprocessing procedures differ substantially across adequacy metrics, ADEPT automatically invokes

the corresponding processing module and generates the intermediate artifacts needed for score computation. This design separates metric-specific implementation details from the external execution workflow and reduces the effort required to reproduce and compare different adequacy techniques.

## 2.3 Cache Management

Many adequacy metrics require intermediate artifacts that are costly to generate but can be reused across repeated evaluations. To avoid redundant computation, ADEPT provides a cache management component that stores and retrieves these reusable artifacts generated during metric-specific processing.

Examples of cached artifacts include neuron profiles for some NC-series metrics, activation traces for LSA/DSA, decision-boundary models for DBC, and generated mutant models for SLMS and MLMS. Cached artifacts are associated with the target model, dataset, metric, and creation time. When compatible cached artifacts are available, ADEPT allows users to choose either to reuse the existing artifacts and proceed directly to scoring, or to regenerate them by invoking the corresponding metric-specific processing module. This mechanism reduces the cost of repeated execution while preserving reproducibility and experimental traceability.

## 2.4 Adequacy Scoring and Reporting

The adequacy scoring component uses the artifacts produced by the processing stage to compute the final score according to the selected metric. Coverage-based metrics report the proportion of covered adequacy elements, LSA/DSA derives scores from activation trace distributions, IDC measures latent feature interaction coverage, DBC quantifies decision-boundary coverage, and SLMS/MLMS reports the classic

mutation score [5], computed as the proportion of killed mutants over all generated mutants, i.e.,  $MS = \frac{|\text{killed mutants}|}{|\text{all mutants}|}$ .

For each run, ADEPT produces the metric adequacy score together with lightweight execution metadata. These metadata include preprocessing time, scoring time, and cache utilization information. The resulting report allows users to inspect the adequacy score, understand the computational cost of each metric, and compare multiple adequacy techniques under a consistent evaluation setting.

Together, these components provide a unified workflow for running, inspecting, comparing, and extending diverse DL test adequacy metrics.

### 3 ADEPT Usage

ADEPT is implemented in Python and can be used through a command-line interface. After checking out the source code from the GitHub repository [7], users can create the required Python environment using the provided `environment.yml` file. The framework currently accepts NumPy (.npz) datasets as input and supports Keras-based DNN models for metrics that require access to the tested model.

Once the environment is set up, users can execute an adequacy metric by specifying the target model, dataset name, test inputs, optional training data, and a metric-specific configuration file. The following command illustrates the general usage pattern:

```
python -m adequacy.cli run \
  --metric [metric-name] \
  --model path/to/model.keras \
  --dataset [dataset-name] \
  --test-data path/to/test_inputs.npz \
  --test-labels path/to/test_outputs.npz \
  --config configs/[metric-name].yaml
```

For metrics that require additional inputs, such as training data for neuron profiling, activation-trace extraction, or mutation-score computation, ADEPT checks the required arguments before execution and reports missing inputs to the user.

Metric-specific options are specified through YAML configuration files. Table 1 summarizes representative configuration parameters for different adequacy metrics in ADEPT. These configuration files allow users to adjust processing and scoring behavior without modifying the source code. If a configuration file is not provided, ADEPT falls back to default values defined in each metric module, enabling users to apply the supported adequacy metrics under common settings. These default configurations are documented in the GitHub repository [7] README.

During processing, ADEPT stores reusable intermediate artifacts in the cache directory. When compatible cached artifacts are available, the framework prompts users to choose whether to reuse the existing artifacts or regenerate them; this choice determines whether the framework skips metric-specific processing and proceeds directly to scoring.

After each run, ADEPT generates a structured JSON report containing the final adequacy score, important configuration, preprocessing time, scoring time, and cache utilization information. These outputs allow users to inspect each run, reproduce adequacy evaluations, and compare multiple metrics under a consistent workflow.

### 4 Related Works

DL test adequacy metrics have been widely studied and can be organized according to classical software testing coverage perspectives [1], depending on the type of signal used to assess test effectiveness.

**Structural adequacy metrics.** Structural-based methods evaluate test adequacy using internal structural features and activation dynamics of the network. NC by Pei et al. [15] and its variants in DeepGauge [12] measure how test inputs activate neurons or cover activation patterns within the network. Beyond single-neuron activation, interaction-based criteria such as DeepCT [13] and MC/DC-style adequacy by Sun et al. [17] extend structural testing principles to neuron interaction relationships, capturing complex inter-dependencies among layer activations.

**Input-space adequacy metrics.** Input-space adequacy metrics estimate test adequacy by characterizing how test inputs cover or deviate within input-related representation spaces. SA [8] evaluates test inputs by measuring their deviation from training data behavior in activation space. These methods quantify input novelty using activation traces and statistical distance measures, reflecting how “surprising” a test input is with respect to previously observed behaviors. IDC [2] assesses test adequacy in learned feature spaces, employing VAE [3, 9] and combinatorial interaction testing over latent features to measure coverage of input feature interactions.

**Boundary-based metrics.** Boundary-based approaches evaluate test adequacy using model uncertainty and decision-region characteristics. DeepBoundary by Liu et al. [11] characterizes test dataset adequacy by modeling the distribution of inputs with respect to the distance to the decision boundary, capturing how close test inputs are to boundary regions.

**Mutation-based metrics.** Mutation-based approaches evaluate test dataset effectiveness by introducing model-level or source-level mutations and measuring whether the test dataset can distinguish the original model from its mutated versions. Representative methods include DeepMutation++ [5], DeepMutation [14], and DeepCrime [6].

Existing available implementations, such as DeepXplore [15], IDC [2], and DeepCrime [6], only focus on specific metrics rather than providing a unified workflow. More broadly, many proposed adequacy metrics remain difficult to run in practice: some lack publicly available implementations, while others depend on broken packages, limited maintenance, or complicated environment



**Table 1.** Representative configuration parameters used by ADEPT.

| Metric                                   | Parameter                           | Metric group | Description                                                                                                                   |
|------------------------------------------|-------------------------------------|--------------|-------------------------------------------------------------------------------------------------------------------------------|
| KMNC, NBC, SNAC<br>All NC-series metrics | train_fraction<br>exclude_layer     | NC-series    | Fraction of training data used for neuron profiling<br>Layer name substrings excluded from profiling                          |
| LSA, DSA<br>LSA                          | layer_names<br>var_threshold        | SA           | Target layers for AT extraction; auto-inferred if omitted<br>Variance threshold for AT dimension filtering before KDE fitting |
| DBC<br>DBC                               | beta<br>max_per_pair                | DBC          | Convergence threshold for bisection-based boundary search<br>Maximum boundary points searched per class pair                  |
| IDC<br>IDC                               | vae_model_dir<br>ways               | IDC          | Path to pre-trained VAE model directory ( <i>required</i> )<br><i>t</i> -way interaction strength for combinatorial coverage  |
| MLMS<br>MLMS                             | num_mutants<br>mode                 | MS           | Number of mutants to generate<br>Execution mode: generate_and_score or score_from_cache                                       |
| SLMS<br>SLMS                             | mutation_operators<br>cache_mutants | MS           | List of mutation operators applied to training pipeline<br>Whether to cache mutant models across runs                         |

setup. A recent empirical study by You et al. [18] further highlights these practical challenges. Practitioners report difficulties in selecting and applying adequacy metrics due to a lack of unified tooling and inconsistent workflows across implementations.

In contrast, ADEPT provides a unified and extensible framework for DL test adequacy evaluation, enabling consistent execution and reuse of representative metrics under a common workflow.

## 5 Tool Availability

ADEPT are publicly available on Zenodo: <https://zenodo.org/records/21682100>. A demo video is also available on YouTube: [https://aub.ie/ADEPT\\_video](https://aub.ie/ADEPT_video)

## 6 Conclusions and Future Work

This paper presents ADEPT, a unified framework for DL test dataset adequacy assessment. The framework integrates representative adequacy metrics in a consistent execution workflow while preserving their metric-specific processing requirements. By providing extensible mechanisms for configuration, metric processing, and reporting, ADEPT simplifies the execution, comparison, and deployment of diverse adequacy techniques. Additionally, by minimizing the engineering effort required to evaluate existing metrics, ADEPT enables researchers and practitioners to easily integrate test adequacy assessment into the development lifecycle of the DL system.

As new adequacy techniques continue to emerge, we envision ADEPT as a continuously evolving platform that facilitates the reproducible evaluation and practical adoption of DL test adequacy assessment methods. As future work, we plan to incorporate more metrics in the framework.

## Acknowledgments

This research is partially supported by the NSF grant #2446393. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the NSF.

## References

- [1] Paul Ammann and Jeff Offutt. 2008. *Introduction to Software Testing*. Cambridge University Press. doi:10.1017/CBO9780511809163
- [2] Swaroopa Dola, Matthew B. Dwyer, and Mary Lou Soffa. 2023. Input Distribution Coverage: Measuring Feature Interaction Adequacy in Neural Network Testing. *ACM Trans. Softw. Eng. Methodol.* 32, 3 (2023), 81:1–81:48. doi:10.1145/3576040
- [3] Irina Higgins, Loïc Matthey, Arka Pal, Christopher P. Burgess, Xavier Glorot, Matthew M. Botvinick, Shakir Mohamed, and Alexander Lerchner. 2017. beta-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24–26, 2017, Conference Track Proceedings*. OpenReview.net. <https://openreview.net/forum?id=Sy2fzU9gl>
- [4] Qiang Hu, Yuejun Guo, Xiaofei Xie, Maxime Cordy, Lei Ma, Mike Papadakis, and Yves Le Traon. 2024. Test Optimization in DNN Testing: A Survey. *ACM Trans. Softw. Eng. Methodol.* 33, 4 (2024), 111:1–111:42. doi:10.1145/3643678
- [5] Qiang Hu, Lei Ma, Xiaofei Xie, Bing Yu, Yang Liu, and Jianjun Zhao. 2019. DeepMutation++: A Mutation Testing Framework for Deep Learning Systems. In *34th IEEE/ACM International Conference on Automated Software Engineering, ASE 2019, San Diego, CA, USA, November 11–15, 2019*. IEEE, 1158–1161. doi:10.1109/ASE.2019.00126
- [6] Nargiz Humbatova, Gunel Jahangirova, and Paolo Tonella. 2021. Deep-Crime: mutation testing of deep learning systems based on real faults. In *ISSTA '21: 30th ACM SIGSOFT International Symposium on Software Testing and Analysis, Virtual Event, Denmark, July 11–17, 2021*, Cristian Cadar and Xiangyu Zhang (Eds.). ACM, 67–78. doi:10.1145/3460319.3464825
- [7] Yidi Kao, Shawn Burnham, Tommi Rose Fahy, and Ali Ghanbari. 2026. <https://zenodo.org/records/21682100>. Accessed: 06/26.
- [8] Jinhan Kim, Robert Feldt, and Shin Yoo. 2018. Guiding Deep Learning System Testing using Surprise Adequacy. *CoRR* abs/1808.08444 (2018). arXiv:1808.08444 <http://arxiv.org/abs/1808.08444>
- [9] Diederik P. Kingma and Max Welling. 2014. Auto-Encoding Variational Bayes. In *2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14–16, 2014, Conference Track Proceedings*, Yoshua Bengio and Yann LeCun (Eds.). <http://arxiv.org/abs/1312.6114>
- [10] Yann LeCun, Yoshua Bengio, and Geoffrey E. Hinton. 2015. Deep learning. *Nat.* 521, 7553 (2015), 436–444. doi:10.1038/NATURE14539
- [11] Yue Liu, Lichao Feng, Xingya Wang, and Shiyu Zhang. 2022. Deep-Boundary: A Coverage Testing Method of Deep Learning Software based on Decision Boundary Representation. In *22nd IEEE International Conference on Software Quality, Reliability, and Security, QRS 2022 - Companion, Guangzhou, China, December 5–9, 2022*. IEEE, 166–172. doi:10.1109/QRS-C57518.2022.00032

- [12] Lei Ma, Felix Juefei-Xu, Jiyuan Sun, Chunyang Chen, Ting Su, Fuyuan Zhang, Minhui Xue, Bo Li, Li Li, Yang Liu, Jianjun Zhao, and Yadong Wang. 2018. DeepGauge: Comprehensive and Multi-Granularity Testing Criteria for Gauging the Robustness of Deep Learning Systems. *CoRR* abs/1803.07519 (2018). arXiv:1803.07519 <http://arxiv.org/abs/1803.07519>
- [13] Lei Ma, Felix Juefei-Xu, Minhui Xue, Bo Li, Li Li, Yang Liu, and Jianjun Zhao. 2019. DeepCT: Tomographic Combinatorial Testing for Deep Learning Systems. In *26th IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER 2019, Hangzhou, China, February 24-27, 2019*, Xinyu Wang, David Lo, and Emad Shihab (Eds.). IEEE, 614–618. doi:10.1109/SANER.2019.8668044
- [14] Lei Ma, Fuyuan Zhang, Jiyuan Sun, Minhui Xue, Bo Li, Felix Juefei-Xu, Chao Xie, Li Li, Yang Liu, Jianjun Zhao, and Yadong Wang. 2018. Deep-Mutation: Mutation Testing of Deep Learning Systems. In *29th IEEE International Symposium on Software Reliability Engineering, ISSRE 2018, Memphis, TN, USA, October 15-18, 2018*, Sudipto Ghosh, Roberto Natella, Bojan Cukic, Robin S. Poston, and Nuno Laranjeiro (Eds.). IEEE Computer Society, 100–111. doi:10.1109/ISSRE.2018.00021
- [15] Kexin Pei, Yinzhi Cao, Junfeng Yang, and Suman Jana. 2017. DeepXplore: Automated Whitebox Testing of Deep Learning Systems. *CoRR* abs/1705.06640 (2017). arXiv:1705.06640 <http://arxiv.org/abs/1705.06640>
- [16] Iqbal H. Sarker. 2021. Deep Learning: A Comprehensive Overview on Techniques, Taxonomy, Applications and Research Directions. *SN Comput. Sci.* 2, 6 (2021), 420. doi:10.1007/S42979-021-00815-1
- [17] Youcheng Sun, Xiaowei Huang, and Daniel Kroening. 2018. Testing Deep Neural Networks. *CoRR* abs/1803.04792 (2018). arXiv:1803.04792 <http://arxiv.org/abs/1803.04792>
- [18] Hanmo You, Zan Wang, Bin Lin, and Junjie Chen. 2025. Navigating the Testing of Evolving Deep Learning Systems: An Exploratory Interview Study. In *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 - May 6, 2025*. IEEE, 2726–2738. doi:10.1109/ICSE55347.2025.00106
- [19] Jie M. Zhang, Mark Harman, Lei Ma, and Yang Liu. 2022. Machine Learning Testing: Survey, Landscapes and Horizons. *IEEE Trans. Software Eng.* 48, 2 (2022), 1–36. doi:10.1109/TSE.2019.2962027

Received 2026-06-25; accepted 2026-07-25